Introdução à Computação
Laboratório 1

Alexandre Rademaker

linha de comando I

Muitos sistemas operacionais modernos têm interfaces gráficas de usuário (GUI) para permitir fácil navegação e manipulação de arquivos e pastas com o mouse.

Ainda assim, como programador, você provavelmente usará a janela do terminal com frequência e poderá realizar muitas das mesmas tarefas através de comandos.

Estes comandos podem ser repetidos e até podemos programar scripts.

linha de comando II

Vamos ver alguns comandos mais comuns no sistemas Linux e MacOS. No Windows também podem ser encontrados no WSL.

O comando `ls` lista os arquivos de uma pasta.

O comando `cd` muda o diretório (pasta) corrente. Os símbolos `.` e `..` correspondem ao diretório corrente e ao diretório pai do corrente. Diretórios podem ser relativos ou absolutos.

```
cd ..
cd ../dir1/dir2/dir3
```

linha de comando III

Podemos criar diretórios como o `mkdir`.

Podemos copiar arquivos e diretórios com o `cp`. Também podemos fazer cópias recursivas com a opção `−r`.

Podemos remover arquivos com `rm`, evitando confirmação com a opção `−f` e para remover diretórios precisamos passar `−r`. E podemos passar os argumentos juntos com `−rf`.

Podemos mover arquivos com `mv` que também recebe origem e destino como parâmetros, podendo ser diretórios ou arquivos.

linha de comando IV

Existem vários outros como: `touch`, `diff`, `rmdir`, `clear`, `chmod` etc.

Use `man CMD` para obter o manual do comando `CMD`, por exemplo, `man ls`.

tipos de dados I

Uma constante ou variábel é basicamente um local nomeado da memória que contém um valor durante a execução do programa.

Os literais são os valores que atribuímos às variáveis ou constantes.

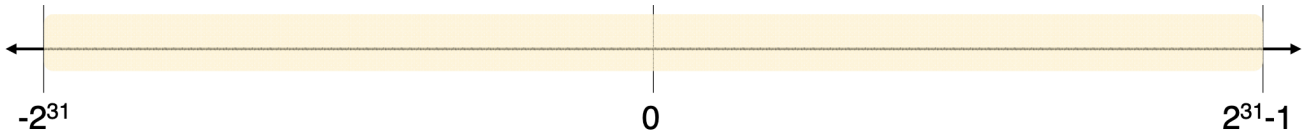
Geralmente, podemos usar os termos literal e constante de forma intercambiável. Por exemplo, na expressão `const int a = 7;` declaramos o nome `a` como uma constante do tipo `int` enquanto nos referimos ao valor 7 como um literal do tipo inteiro.

Os literais `3.4` ou `3.4E4` são valores do tipo ponto-flutuante. E o literal `'a'` é um caracter.

tipos de dados II

O tipo `int` é usado para armazenar inteiros.

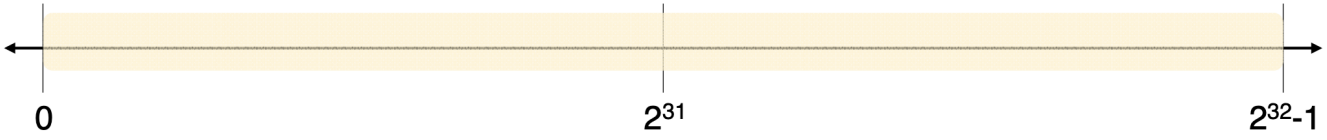
Inteiros sempre ocupam 4 bytes de memória (32 bits). Isso significa que o intervalo de valores que eles podem armazenar é necessariamente limitado a 32 bits de informação.



tipos de dados III

`unsigned` é um qualificador que pode ser aplicado a certos tipos (incluindo `int`), o que efetivamente dobra o intervalo positivo desse tipo, ao custo de não permitir valores negativos.

```
1 unsigned int a = 100;
```

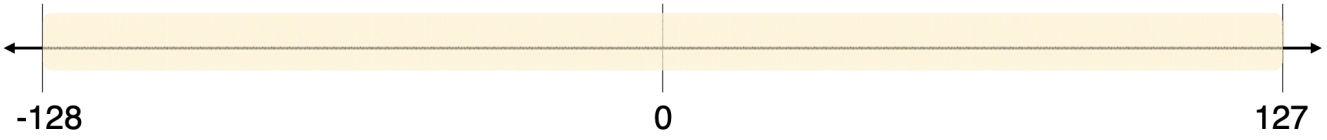


tipos de dados IV

O tipo de dados `char` é usado para variáveis que armazenam caracteres.

Os caracteres sempre ocupam 1 byte de memória (8 bits). O intervalo de valores que eles podem armazenar é necessariamente limitado a 8 bits de informação.

A tabela ASCII, oferece um mapeamento de caracteres como A, a, B, C, etc... para valores numéricos no lado positivo deste intervalo.



tipos de dados V

O tipo de dados `float` é usado para variáveis que irão armazenar valores de ponto flutuante, números reais.

Valores de pontos flutuantes sempre ocupam 4 bytes de memória (32 bits).

Com 32 bits de precisão, alguns dos quais podem ser usados para uma parte inteira, estamos limitados em quão precisos podemos ser. Mais aqui.

tipos de dados VI

O tipo de dados `double` é usado para variáveis que irão também armazenar valores de ponto flutuante. A diferença é que doubles são precisão dupla. Eles ocupam sempre 8 bytes de memória (64 bits).

Com 32 bits adicionais de precisão em relação a um `float`, os doubles nos permitem especificar números reais muito mais precisos.

tipos de dados VII

O tipo `void` é um tipo, mas não um tipo de dados.

As funções podem ter um tipo de retorno `void`, o que significa apenas que não retornam um valor.

A lista de parâmetros de uma função também pode ser `void`. Significa simplesmente que a função não aceita parâmetros. Um substituto para 'nada'.

tipos de dados VIII

O tipo `bool` contém apenas dois valores: `true` e `false`.

Para usar estas constantes, devemos importar a biblioteca `stdbool.h` com a macro `#include <stdbool.h>`.

tipos de dados IX

Também encontraremos estruturas (`structs`) e tipos definidos (`typedef`) que oferecem grande flexibilidade na criação dos tipos de dados necessários para seus programas.

Variáveis I

C exige que você especifique o tipo de dados de cada variável, na criação da variável com ou sem sua inicialização.

Variáveis podem ser declaradas sem serem inicializadas.

```
1 int a = 10;
2
3 int a;
4 a = 10;
```

Variáveis II

Se você deseja criar várias variáveis do mesmo tipo, especifique o nome do tipo uma vez e, em seguida, liste quantas variáveis desse tipo desejar.

```
1 int height, width;
2 float sqrt2, sqrt3, pi;
```

Em geral, é recomendável declarar variáveis apenas quando necessário.

Operadores I

Para manipular variáveis e valores em C, precisamos de operadores. Os operadores permitem construir valores a partir de outros valores.

Em C temos a adição (`+`), subtração (`−`), multiplicação (`*`) e divisão (`/`) de números, operadores aritméticos. Também temos o módulo (resto da divisão, `%`).

```
1 int x = y + 1;
2 x = x * 5;
3 int m = 13 % 4;
```

Operadores II

Também podemos usar algumas formas abreviadas (açucares sintáticos) para combinar expressões com a atualização de valores em variáveis.

```
x = x * 5;
x *= 5;

x = x + 1;
x++;

x = x - 1;
x--;
```

Operadores III

Expressões booleanas são usadas em C para comparar valores.

Todas as expressões booleanas em C são avaliadas como um dos dois valores booleanos possíveis – verdadeiro ou falso.

Podemos usar o resultado da avaliação de uma expressão booleana em outras construções de programação, como condicionais.

Operadores IV

Às vezes, ao trabalhar com expressões booleanas, usaremos variáveis do tipo bool, mas nem sempre.

Em C, todo valor diferente de zero é equivalente a `true` e zero é `false`.

Expressões booleanas são construídas com operadores lógicos e operadores relacionais.

Operadores V

O AND lógico (`&&`) é verdadeiro se e somente se ambos os operandos forem verdadeiros, caso contrário, falso.

x	y	x && y
true	true	true
true	false	false
false	true	false
false	false	false

Operadores VI

O OR lógico (`||`) é verdadeiro se e somente se pelo menos um operando for verdadeiro, caso contrário, falso.

x	y	x || y
true	true	true
true	false	true
false	true	true
false	false	false

O NOT lógico (`!`) inverte o valor de seu operando.

Operadores VII

Os operadores relacionais se comportam como você esperaria e parecem sintaticamente semelhantes a como você pode recuperá-los da aritmética elementar.

- Less than (`x < y`)
- Less than or equal to (`x <= y`)
- Greater than (`x > y`)
- Greater than or equal to (`x >= y`)

Operadores VIII

Também podemos testar igualdade (`x == y`) ou desigualdade (`x != y`).

Cuidado! É um erro comum usar o operador de atribuição (`=`) quando você pretende usar o operador de igualdade (`==`).

Condicionais I

As expressões condicionais permitem que seus programas tomem decisões e façam diferentes bifurcações no fluxo de execução, dependendo de testes lógicos.

C fornece algumas construções diferentes para implementar condicionais.

```
1 if (boolean-expression) {
2   // code
3 }
```

Condicionais II

Também podemos especificar um bloco para ser executado caso a expressão booleana seja falsa.

```
1 if (boolean-expression) {
2   // code
3 }
4 else {
5   // code
6 }
```

Condicionais III

E podemos criar uma cadeia de testes mutuamente exclusivos.

```
if (boolean-expression1) {
  // code
}
else if (boolean-expression2) {
  // code
}
else if (boolean-expression3) {
  // code
}
else {
  // code
}
```

Condicionais IV

Ou não mutuamente exclusivos.

```
1  if (boolean-expression1) {
2    // code
3  }
4  if (boolean-expression2) {
5    // code
6  }
7  if (boolean-expression3) {
8    // code
9  }
10 else {
11   // code
12 }
```

Condicionais V

Também temos o operador `switch` que permite enumerar casos para uma data expressão. Observe os `break`.

```
int x = <valor>;
switch(x) {
  case 1:
   printf("one!");
   break;
  case 2:
   printf("Two!");
   break;
  case 3:
   printf("Three!");
   break;
  default:
   printf("Sorry!");
}
```

Condicionais VI

Finalmente, também podemos escrever uma expressão com o operador ternário `?:`. Note que blocos de apenas uma instrução não precisam das chaves.

```
int x = (bool-expr) ? 5 : 6;

int x;
if (bool-expr)
  x = 5;
else
  x = 6;
```

Loops ou repetições I

while use quando quiser que um loop seja repetido um número desconhecido de vezes e, eventualmente, nenhuma.

do-while use quando quiser que um loop seja repetido um número desconhecido de vezes, mas pelo menos uma vez.

for use quando quiser que um loop se repita um número discreto de vezes, embora você possa não saber o número no momento em que o programa é compilado.

Loops ou repetições II

Ao encontrar o comando `for`, o programa irá executar o comando `start`. A expressão `expr` é avaliada. Se for verdadeira, o bloco é executado. Se for falsa, o bloco não é executado e o fluxo passa para o próximo comando depois do `for`. Se o bloco for executado, após ele, o comando `increment` é executado e novamente `expr` é avaliada e o ciclo continua.

```
1 for (start; expr; increment) {
2   // code
3 }
```

Loops ou repetições III

No primeiro caso o bloco pode não ser executado nenhuma vez. No segundo, pelo menos a primeira vez ele será executado.

```
1 while (boolean-expr) {
2   // code
3 }
4
5 do {
6   // code
7 } while (boolean-expr)
```

números mágicos I

Constantes em nosso código às vezes são chamadas de números mágicos.

Imagine que seu código depende de certas constantes que, a princípio, você não pensa em mudar. Eventualmente, este número poderá ter que ser usado em mais de um local.

Imagine que em algum momento no futuro, você precisa alterar este valor. Agora terá que procurar em todos os trechos do código para não introduzir um bug.

números mágicos II

Imagine um programa manipulando cartas de um baralho. Outras funções deste programa provavelmente também precisarão da constante 52.

```
card deal_cards(deck name) {
  for (int i = 0; i < 52; i++) {
    // deal the card
  }
}
```

números mágicos III

Podemos usar uma macro para que o pré-processador (antes do compilador) substitua ocorrências de um nome pela expressão associada.

Se `#include` é como copiar/colar, o `#define` é análogo ao find/replace.

```
1 #define DECKSIZE 52
2
3 card deal_cards(deck name) {
4   for (int i = 0; i < DECKSIZE; i++) {
5     // code
6   }
7 }
```

números mágicos IV

Um outro exemplo bem prático é para definir constantes que possamos precisar em várias partes do programa.

```
1 #define PI 3.14159265
```

Geralmente declaramos estas diretivas no topo dos arquivos `.c` e geralmente usamos nomes em maiúsculas.